

HBase on S3

BD&O VAT 2023.06.13
Andrew Purtell

Summary Brief

The Salesforce Phoenix service* is the combination of Apache HBase, Phoenix, Hadoop, ZooKeeper, and many additional internal components into a complete System of Record, a horizontally scaled relational non-transactional datastore, operating on all substrates and in all first party data centers, delivering consistent low latency responses at 99.95% availability generally and 99.999% for Vagabond, with no dependencies on Core, underpinning customer facing products generating hundreds of millions of dollars of yearly ACV. HBase without Phoenix also provides persistence for Argus.

Salesforce Phoenix is on a similar journey as SDB onto Amazon's S3 and other S3-alike object stores supporting the API, seeking more cost effective persistence options for Hyperforce, so we can offer differentiated cost models that will unlock more opportunities for our customers.

* - For more detail please see the [Salesforce Phoenix Service Architecture "One Pager"](#)

Definitions

HBase: the storage and low level query engine of the Salesforce Phoenix service.

Phoenix: a JDBC driver and relational data management framework built on top of HBase

HDFS: the Hadoop Distributed File System, a horizontally scale out file system for large distributed data-intensive applications, providing persistence for HBase.

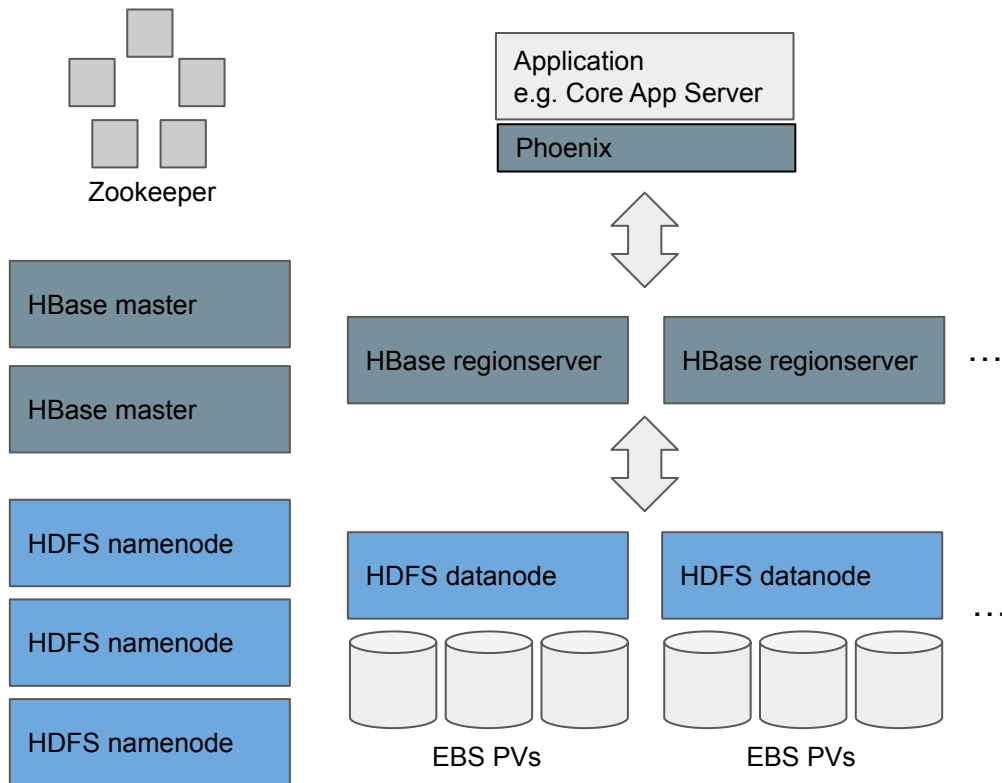
HFile: the HBase on disk file storage format, the files where data committed to HBase is persisted and managed in long term storage.

WAL: write ahead log, a file based technique for providing atomicity and durability of commits in database systems, the files where data committed to HBase is temporarily stored until durably recorded into HFiles.

EBS: Amazon Web Service's Elastic Block Storage service, a block storage service designed to emulate network attached disk devices for Elastic Compute Cloud (EC2) applications.

S3: Amazon Web Service's Simple Storage Service, the cloud object storage service designed for EC2 applications.

Current Hyperforce Deployment Architecture



The Phoenix driver coordinates data persistence and access with the HBase master and regionservers.

HBase regionservers delegate low level data persistence and access functions to the HDFS namenode and datanodes, for both WAL and HFiles.

HDFS datanodes are attached to encrypted Elastic Block Service (EBS) physical volumes.

HDFS triple replicates WAL and HFile data and guarantees replicas in three AZs.

Why HDFS rather than S3 from the beginning?

Primarily as a risk management strategy we opted for HDFS for the first generation of our service's Falcon architecture. Our datastore underpins customer facing products generating hundreds of millions of dollars of yearly ACV. We need to be careful.

HDFS provides a well known persistence option with capabilities and performance we have 10 years of experience with. Shifting our first party HDFS based persistence architecture into Falcon allowed us to guarantee the same performance in Falcon as customers experienced in first party and baked as expectation into their products.

Existing HDFS architecture and features were able to support a 3-AZ deployment configuration without significant changes. There were many aspects of our operations we needed to innovate for Falcon, but not HDFS.

Why S3 now (or, what's the issue with HDFS)?

In three words: cost to serve. In three letters: EBS

As with many other services, our service's initial Falcon deployment architecture is complete, and deployed, and in production serving customer workloads... and now the monthly bills are coming in.

Highly granular metered infrastructure and detailed billing is providing for the first time in Salesforce history the ability to deep dive into the various services' and products' operating expenses and account specific and accurate costs to dependencies.

We have a company-wide mandate to increase operating efficiency.

Naturally, our customers and ourselves want to lower our operating expenses, where alternatives are available and reasonable, where we will not sacrifice on Trust.

HBase on S3 Project Goals

- Salesforce Phoenix continues to provide a highly available service offering low latency data access backed by HDFS on EBS. **The status quo does not change.** Incurred costs are charged back at a “standard cost” of \$x/GB.
- Customers can opt in (can be with per-table granularity) to a **less expensive level of service** backed by S3 with the same availability but with a **different performance profile**. The lower charge- or share-back price unlocks new opportunities. Incurred costs are charged back at \$y/GB where $y < x$.
- [*Stretch goal*] Customers can opt in to the S3 backed option for their tables and, additionally, can opt in to an **individually provisioned and metered in-memory cache**. Some savings compared to the status quo are still possible and the cache guarantees continued low latency access for the configured working set. Incurred costs are charged back at $y/GB + c/GB$ where $y < x$ and c is the additional cost for the allocated cache.

HBase on S3 Project Goals (cont.)

- Incremental data migration.
 - Tenants can opt-in to S3 backed tables on a per table basis. Migration of existing tables from HDFS to S3 will be managed table by table.
- Transparent data migration.
 - Seamless background migration from existing HDFS backed tables to S3 backed tables will be enabled by Phoenix's online data format change capabilities.
 - No loss of Phoenix functionality during and after the migration.

Hypothetical Service Models

1) Standard

WAL on HDFS/EBS with triple replication. Shared with HFiles.

HFiles on HDFS/EBS with triple replication.

Best effort in memory block cache.

Requires some EBS level storage reservation. *

2) Reduced Cost

WAL on HDFS/EBS with triple replication.

HFiles on S3, at least 80% less expensive by \$/GB/month b/c no replicas required. S3 has internal redundancy.

Best effort in memory block cache.

S3 is fully metered usage, no reservation required.

Different performance profile.

3) Reduced Cost + Cache

WAL on HDFS/EBS with triple replication.

HFiles on S3, at least 80% less expensive by \$/GB/month b/c no replicas required. S3 has internal redundancy.

Block cache provisioned for an extra \$/GB per allocated cache resources.

S3 is fully metered usage, no reservation required.

Different performance profile.

* - EBS volumes must be allocated all at once before attachment and utilization.

HBase on S3 Project Non-Goals

- Elimination of HDFS.
 - We will continue to meet today's expectations of a certain low latency performance profile.
 - S3 and S3-alike cloud object stores, except for Azure's ABFS, cannot support HBase write ahead logs (WALs). *
- Caching of the entire data set.
 - SDB's current S3 enabled architecture caches 100% of the data they store in S3 in an external in-memory cache in order to guarantee low latency access to any item at any time**. This is quite expensive and a significant challenge to achieving cost savings. In contrast, if we decide to offer something other than best effort undifferentiated caching, we plan to offer tenants a variable and tunable amount of per-table data caching. What a tenant pays for will be reserved for them and they will determine the cost-benefit tradeoffs of provisioning options.

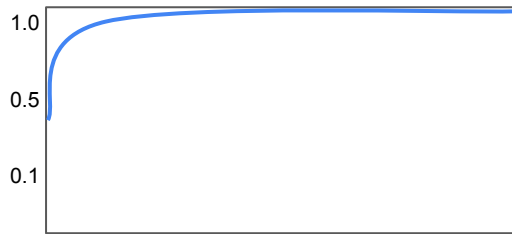
* - Long term we can look at alternatives to HDFS for WAL on substrates where ABFS is not available, such as BookKeeper, and then eliminate it from our deployment architectures everywhere.

** - Oracle's single block read latency is < 10ms.

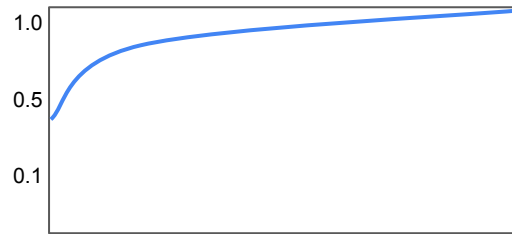
Expected Performance Differences

S3 backed tables will trade off reduced costs for a different level of performance. We can continue to expect mean low latencies due to HBase always providing a block cache (if only undifferentiated and best effort), but, because complete caching of all data is a non-goal, for the cache miss cases there will be increased end user response latencies*.

Metered per-tenant caches can be a tool for lowering latencies in the high percentiles if and only if the use case requires it (and will pay for it), recovering S3 related performance losses.



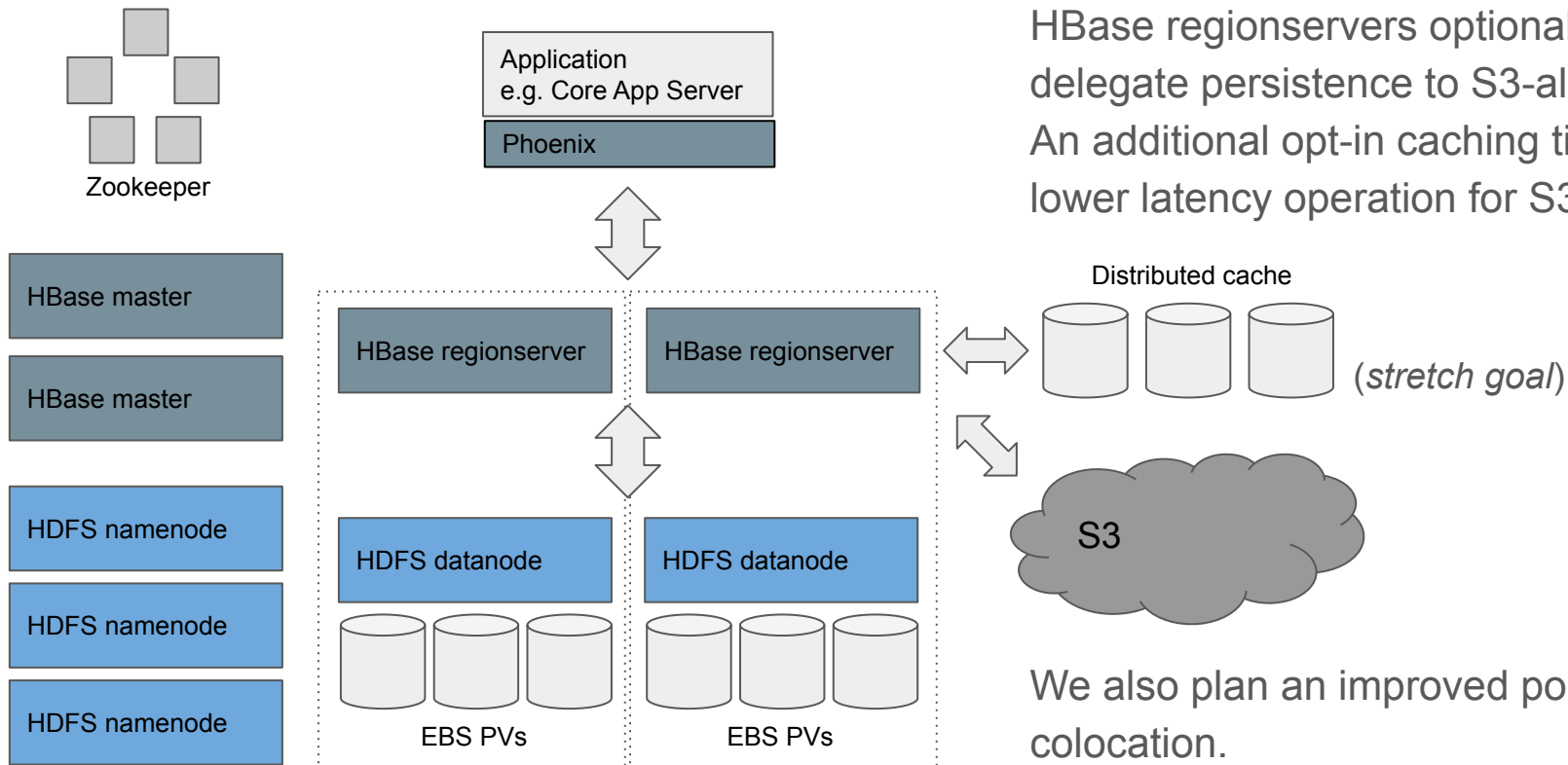
Hypothetical HDFS backed table
operation latencies, plotted as CDF



Hypothetical S3 backed table
operation latencies, plotted as CDF

* - In some simple tests, cache miss reads from S3 tables average < 1 s but can be several seconds.

S3 Enabled Hyperforce Deployment Architecture



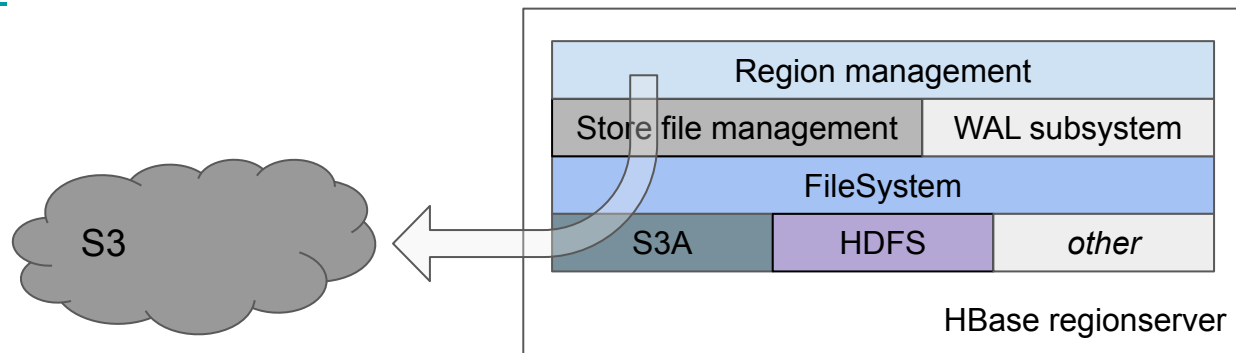
HBase regionservers optionally delegate persistence to S3-alike stores. An additional opt-in caching tier enables lower latency operation for S3 tables.

We also plan an improved pod level colocation.

What Open Source HBase Provides Out of The Box

HBase delegates persistence to the Hadoop [FileSystem](#) layer. Hadoop ships implementations of FileSystem for the Hadoop native DistributedFileSystem (HDFS), Azure's ABFS and Data Lake, Amazon's S3 and other S3-alike stores, a mount table like aggregator of child filesystems (ViewFileSystem), and a few other more esoteric options.

HBase leverages Hadoop's [extensive S3 support](#): [authentication](#), [credential management](#), [retry and recovery strategies](#), [storage class tiering](#), [request parallelization and performance optimization](#), and [metrics](#).



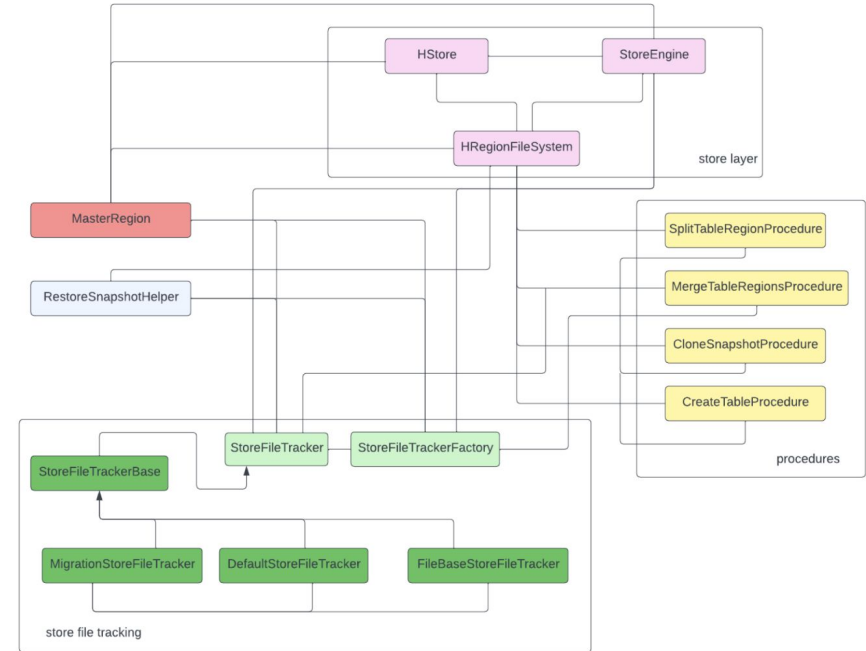
What Open Source HBase Provides (cont.)

StoreFileTracker

New interfaces in 2.5+ that indirect the listing, creation, movement, and deletion of store files.

New site configuration options for choosing the type of store file tracker. Can be configured on a per table basis.

Automatic and transparent migration of store file management from one implementation to another.



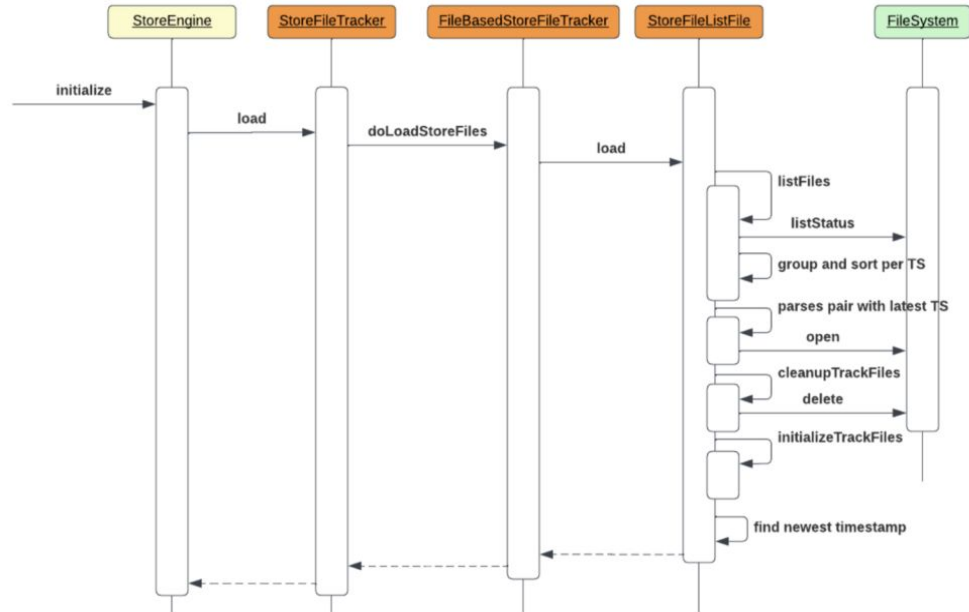
What Open Source HBase Provides (cont.)

FileBasedStoreFileTracker

Does not assume the filesystem (HDFS or S3) can provide atomic metadata operations.

Encodes the store's directory as a protobuf structured manifest that can be atomically read and updated on all substrates.

Does not yet fully cover all possible operations that should be atomic or virtual, like store archiving and file links.



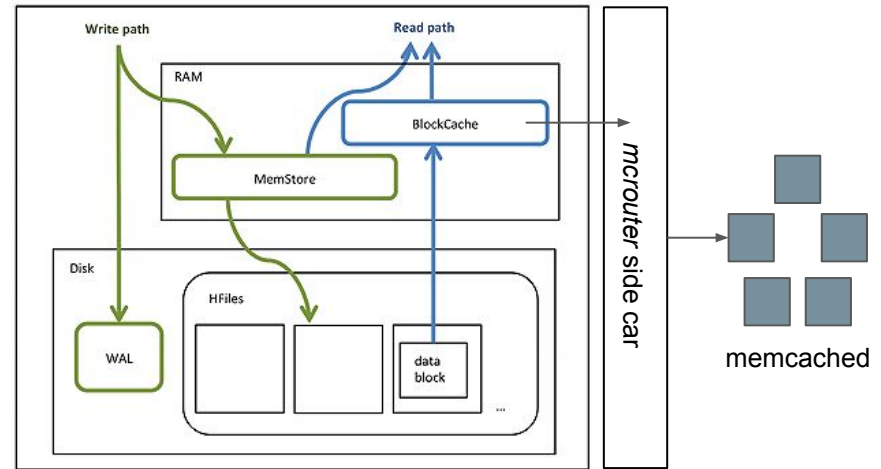
What Open Source HBase Provides (cont.)

External block cache (L2)

HBase can use any in-memory cache solution that supports the memcached protocol.

Instagram's [mcrouter](#) deployed as a sidecar can provide sharding and consistent hashing over a set of remote memcached servers.

We can modify HBase to support different block cache configuration by table. This will let us associate dedicated memcached clusters with a tenant's tables, simplifying the problem of how to reserve resources and show back cache costs by tenant.



Transparent Data Migration from EBS to S3

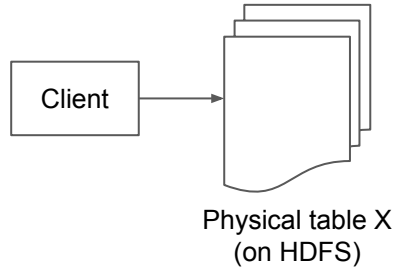
Phoenix provides recent [support](#) for online data format changes. Physical HBase tables are abstracted from the logical view over table structure that Phoenix metadata provides. A triggering event, an ALTER command, starts a transformation process that is tracked and managed using a system table in a way that is transparent to the user.

The transformation mechanism and its performance impact are very similar to the secondary index rebuild process and leverages much of its machinery.

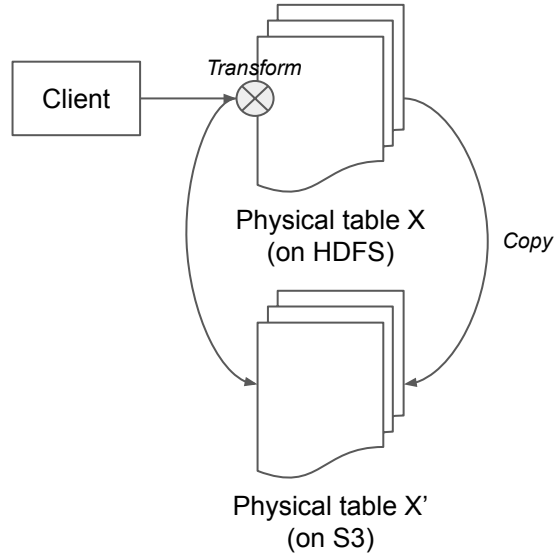
We will use this Phoenix feature to transparently migrate existing physical tables backed by HDFS to new physical tables backed by S3 without a loss of service and without compromising performance.

Notably, because it is a Phoenix feature, transparent migration will not be available for Argus, which uses a bespoke HBase client to interact with HBase directly. The migration process that Argus would implement is the same: new table in S3, write to new table, read from both, copy over historical data if desired, eventually cut over to the new table.

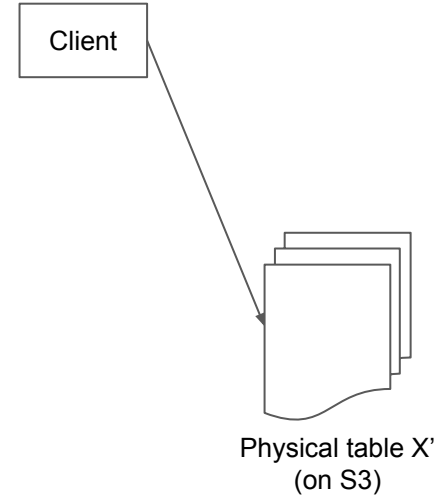
Transparent Data Migration from EBS to S3 (cont.)



Before ALTER ASYNC



ALTER ASYNC in progress



ALTER ASYNC completed

Current Status

- We are testing and validating what HBase provides out of the box:
 - *StoreFileTracker* interface and *FileBasedStoreFileTracker*, which tracks live store files and not-yet-live flushes and compaction temporaries using a manifest file, avoids issues where cloud object stores do not provide atomic semantics, but is not yet feature complete for us.
 - External blockcache, a block cache option that will store and retrieve block data from anything that can speak the memcached wire protocol, seems something we can work with.
- We have identified some gaps and insufficiencies and have planned fixes for them:
 - We cannot presume HDFS's low impact open file accounting is in effect ([HBASE-20430](#)).
 - The *FileBasedStoreFileTracker* is not feature complete, missing indirection and optimizations for splitting and merging ([HBASE-27826](#)) and file archiving ([HBASE-27825](#)).
 - HBase's "file" paths are a performance anti-pattern for S3 ([HBASE-27841](#)).
 - For more detail refer to the [HBase on S3 "One Pager"](#).
- We are developing a model that will, given metrics from production HBase clusters, estimate the costs if the same data shape and workload were to be backed by S3, or S3 + cache tier.

Current Status (cont.)

- We need to refresh our Hyperforce deployment manifests and scripts so we can renew stability and performance testing and development iteration of small S3 backed dev clusters using our latest stack HBase.13 (Hadoop 3.3 + HBase 2.5).
 - So far testing has been confined to single node development environments. That has been very useful but the low hanging fruit is mostly resolved now.
 - Until recently this investment would have been premature but open source has evolved.
 - The Hadoop 3.3 code line is the first where the S3A filesystem is considered GA by the community.
 - The HBase 2.5 code line is the first to ship StoreFileTracker.

Next Steps

- For differentiated per table file placement:
 - HBase currently supports configuration of only one global setting for HFiles placement and another global setting for WAL file placement. There are many places in the code where objects and configuration are cached with this in mind. We need to undo these assumptions.
 - After our modifications are completed table schemas in HBase will be able to specify per-table overrides of table data placement. The global configuration options will still be supported and used as default.

```
hbase> CREATE 'myTable', { NAME => 'myFamily', COMPRESSION => 'ZSTD', ... }, ... ,  
          CONFIGURATION => { 'hbase.store.dir' => 's3://myBucket/hbase' }
```

or, with some syntactic sugar:

```
hbase> CREATE 'myTable', { NAME => 'myFamily', COMPRESSION => 'ZSTD', ... }, ... ,  
          STORE_URI => 's3://myBucket/hbase'
```

Next Steps (cont.)

- For differentiated per table file placement:

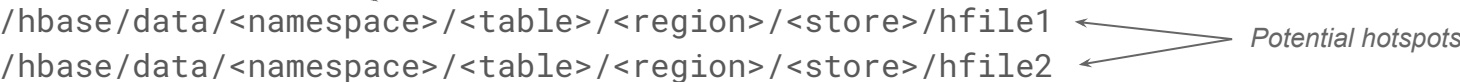
```
hbase> CREATE 'myTable',  
      { NAME => 'familyOnS3', COMPRESSION => 'ZSTD',  
        'hbase.store.dir' => 's3://myBucket/hbase' },  
      { NAME => 'familyOnHDFS', COMPRESSION => 'LZ4',  
        'hbase.store.dir' => 'hdfs://myNamespace/hbase' }
```



In HBase schema, store level options can be configured with column family granularity, so our planned modifications will technically support HFile placement on anything supported with a Hadoop FileSystem implementation **by column family**, although we expect Phoenix will stick to table level differentiation.

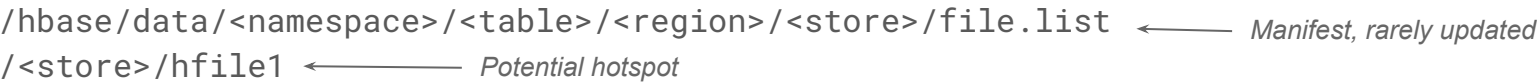
Next Steps (cont.)

- For avoiding an S3 specific performance anti-pattern:
 - S3 throttles IO to buckets after 3,500 PUT, COPY, POST, or DELETE or 5,500 GET or HEAD requests per second per prefix. Prefix partitioning is a black box to us and cannot be hinted or controlled. Today's HFile on S3 support lays out "files" in the S3 bucket exactly as it does in HDFS. We have a fan-out path structure that all but guarantees we will be throttled at an inconvenient choke point for what would otherwise be a stochastically even distribution of load over the leaves.

Potential partition points and bottlenecks
 - We have:

/hbase/data/<namespace>/<table>/<region>/<store>/hfile1
/hbase/data/<namespace>/<table>/<region>/<store>/hfile2

Potential hotspots

...
 - We need:

/hbase/data/<namespace>/<table>/<region>/<store>/file.list
/<store>/hfile1

Manifest, rarely updated
Potential hotspot

...
 - Region and store names are cryptographic hashes so by placing this high entropy component near the left leading edge of the bucket key we minimize the chance of inopportune partitioning.

Next Steps (cont.)

- For the optional caching related stretch goals:
 - After some bug fixing the external block cache option functions well in single server testing but clearly has not been used in production for some time, if ever.
 - Refactor of how block caches are allocated and managed to support per table block cache settings and multiple block cache instances coexisting in the same regionserver.
- For leveraging Phoenix's online format change for transparent migration of HFile data:
 - Introduce the concept of *storage engine*, [in the MySQL sense](#), into the Phoenix grammar as a table attribute, e.g.
 - `CREATE TABLE ( ... ) ENGINE=S3`
 - `ALTER TABLE ... ASYNC ENGINE=S3`
 - `ALTER TABLE ... ASYNC ENGINE=HDFS`
 - Map Phoenix ENGINE attribute values to equivalent HBase level schema details.
 - Register a change of the storage engine table attribute as a qualifying event for online schema migration.

Next Steps (cont.)

- For the [HBASE-20430](#) problem:
 - Today HBase holds open all active store files. This is so the embedded HDFS client will cache block locations locally and we don't need to round trip to the namenode first for that metadata every time we read from the file. HDFS makes this extremely cheap.
 - The S3A client is pretty smart about how it multiplexes requests for N resources in a S3 bucket over a smaller pool of M dedicated HTTP request threads, but we need to ensure that we do not accidentally inflate demands on S3A's fixed resource pools.
 - This probably means we will change how HBase manages open store files: by capping the number of concurrent open files, by introducing a new managed open file cache, and by managing the cache on an LRU or LFU basis.

Next Steps (cont.)

- Bringing it all together:
 - Let's call our S3 enabled release **HBase.14**.
 - Deploy HBase.14 to Argus and Commercial pre-production environments for stability and performance testing, and for iterating on any necessary final improvements along the path to GA.
 - Identify and onboard pilot use cases for S3 backed tables.
 - Especially useful will be those that involve a migration of existing data out of HDFS.
 - In production pilots.
 - Worldwide go-live.

Thanks and Q&A